Lecture 4.2 – PI Tuning and Anti-Windup

Week 4: Making PI Control Work Reliably on the Motor

Learning Objectives

By the end of this lecture, you should be able to:

- compare P and PI control using measured data;
- explain the effect of increasing K_i;
- identify signs of poor PI tuning;
- apply a safe PI tuning procedure;
- explain integral clamping and anti-windup;
- use Python plots to compare PI tests.

1. The Main Goal of Week 4

Week 3 showed that P control can reduce speed error but usually leaves some steady-state error.

Week 4 adds integral action so the controller can continue correcting small persistent errors.

The goal is not to make the controller complicated.

The goal is to make the motor speed track the reference better while keeping the response smooth and safe.

> Week 4 is about improving tracking accuracy without making the motor response rough or unstable-looking.

2. PI Control Review

The PI controller is

$$u(t) = K_p e(t) + K_i \int e(t)\, dt.$$

In discrete time,

$$I[k] = I[k-1] + e[k]T_s$$

and

$$u[k] = K_p e[k] + K_i I[k].$$

For Arduino implementation:

$$T_s = 0.10 \text{ s}$$

is a reasonable starting point.

3. How K_i Changes the Response

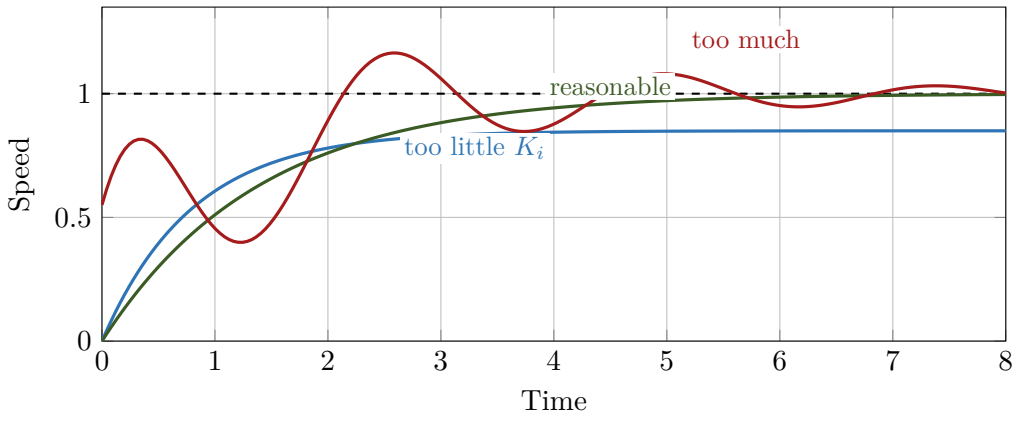
With K_p fixed, increasing K_i usually:

- reduces steady-state error;
- increases the controller's memory of past error;
- may increase overshoot;
- may increase oscillation;
- may increase time spent in PWM saturation.

A larger K_i is not always better. Too much integral action can make the response worse.

4. Comparing Three Integral Gains

Assume the same K_p, but different K_i values.



5. Signs That K_i Is Too Small

A small K_i may show:

- steady-state error remains noticeable;
- speed slowly creeps toward the reference;
- disturbance recovery is slow;
- response looks very similar to P control.

This is safe, but the tracking improvement may be limited.

6. Signs That K_i Is Too Large

A large K_i may show:

- overshoot above the reference;
- oscillation around the reference;
- PWM stays at 255 for too long;
- motor sounds rough;
- settling time becomes longer instead of shorter.

If increasing K_i makes the response overshoot or oscillate more, reduce K_i.

7. Integral Windup in the Lab

During startup, the motor may be far below the desired speed, so the error is large.
The controller may command maximum PWM:

$$u = 255.$$

While this happens, the integral term may keep increasing.
If it grows too much, the motor may overshoot after it finally reaches the reference.
This is why we use integral clamping.

8. Integral Clamping

Limit the integral state:

$$I_{\min} \leq I \leq I_{\max}.$$

In code:

```
integral = integral + error * Ts;

integral = constrain(integral, integralMin, integralMax);
```

A simple starting value is to choose a clamp that keeps the integral contribution from becoming too large:

$$|K_i I| \leq 100 \text{ to } 150 \text{ PWM units.}$$

This is only a practical rule, not a universal law.

9. Safe PI Tuning Procedure

Use this procedure in Lab 4:

1. Use the selected K_p from Week 3.
2. Run one baseline P-control test with $K_i = 0$.
3. Choose a small K_i.
4. Run a PI test and save the data.
5. Increase K_i moderately.
6. Compare steady-state error, overshoot, settling behavior, and maximum PWM.
7. Stop increasing K_i if the response becomes oscillatory or rough.

> Only change one tuning parameter at a time. Keep K_p fixed while testing different K_i values.

10. What to Measure

For each test, record:

Quantity	Reason
Reference RPM	target speed
Steady-state RPM	tracking result
Steady-state error	accuracy
Overshoot	too much integral action
Settling time	response speed
Maximum PWM	saturation/effort
Integral value	windup behavior

11. Python Data Analysis

The Arduino should send:

time, reference, rpm, error, pwm, integral

Python saves this to CSV and creates plots:

- reference and RPM versus time;
- PWM command versus time;
- integral state versus time;
- comparison of several K_i values.

12. P vs. PI Comparison

The final lab comparison should include:

- one P-only response;
- at least two PI responses;
- a table comparing error, overshoot, and saturation;
- a selected PI tuning.

A good PI tuning is not only the one with the smallest final error. It should also be smooth, repeatable, and safe for the hardware.

13. Simple PI Selection Rule

For this class, a reasonable selected PI controller should:

- reduce steady-state error compared with P control;
- avoid large overshoot;
- avoid long saturation;
- settle in a reasonable time;
- produce a smooth PWM signal;
- behave similarly across repeated trials.

14. Practice Problem

A team uses

$$r = 120 \text{ RPM}, \qquad K_p = 3.$$

They test three integral gains.

K_i	Final RPM	Overshoot	Max PWM	Observation
0	102	none	180	large error
0.2	118	small	220	smooth
1.0	124	high	255	oscillatory

Which K_i would you choose and why?

15. Connection to Lab 4

In Lab 4, you will:

1. implement PI control on Arduino;
2. record motor data using Python;
3. compare P and PI responses;
4. test several K_i values;

5. observe integral windup and clamping;
6. select a final PI controller for your motor.

Summary

- PI control adds accumulated error to proportional control.
- Integral action reduces steady-state error.
- Too much K_i can create overshoot and oscillation.
- Integral windup occurs during saturation.
- Integral clamping is a simple anti-windup strategy.
- Python plots help compare controller performance.
- A good PI controller balances accuracy, smoothness, and safety.